



LINFO 1121
DATA STRUCTURES AND ALGORITHMS



TP1: Complexités, piles, Files et listes

Question 1.1.1: Qu'est-ce qu'un type abstrait de données?

En informatique, un **type abstrait** est une spécification mathématique d'un ensemble de **données** et de l'ensemble des opérations qu'on peut effectuer sur elles. On qualifie d'**abstrait** ce **type** de **données** car il correspond à un cahier des charges qu'une structure de **données** doit ensuite implémenter.

Question 1.1.1: Qu'est-ce qu'un type abstrait de données?

**DE QUOI SE RAPPROCHE LE PLUS UN
TYPE ABSTRAIT DE DONNÉE EN JAVA?**

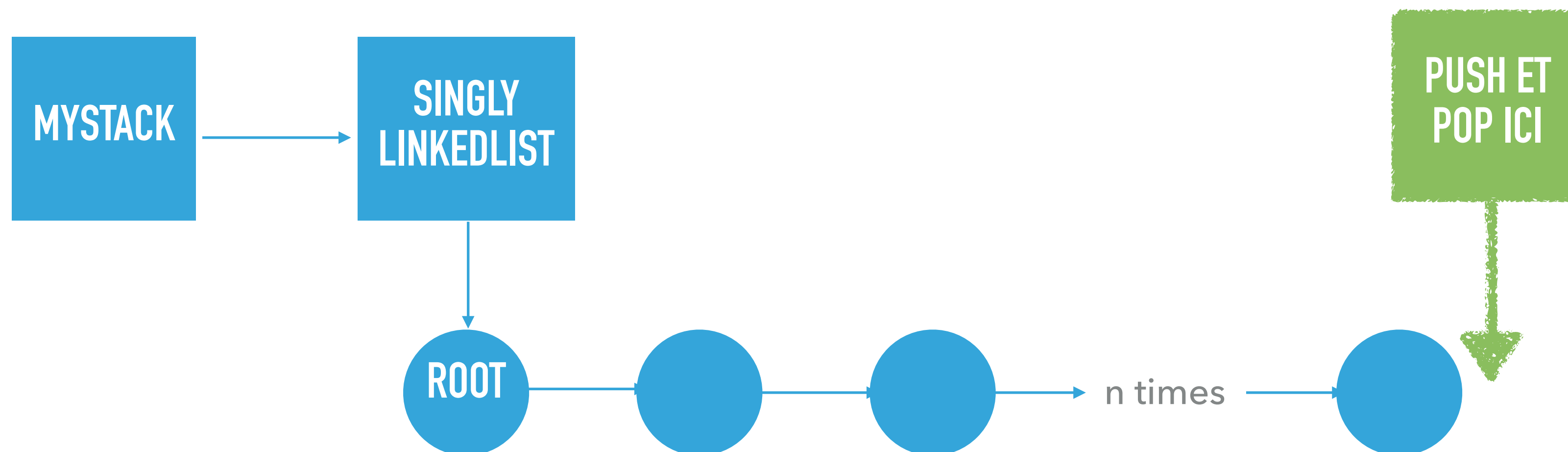
- Une classe
- Un objet
- Une interface
- Une structure
- Un type primitif

1.1.2 Stack avec liste chaînée

Implémentation d'une pile (stack) avec une liste simplement chaînée, où les opérations se font en fin de liste?

Complexité d'un push/pop ?

$\mathcal{O}(1)$	~ 1	$\Omega(1)$	$\Theta(1)$
$\mathcal{O}(n)$	$\sim n$	$\Omega(n)$	$\Theta(n)$



**QUESTION 1.1.3: QUELLES SONT LES
IMPLÉMENTATIONS POSSIBLES POUR
UNE PILE?**

Dans la version de Java ?

Pourquoi `java.util.Stack` est-il implémenté via un tableau (extension de `Vector`) ?

Est-ce si inefficace que cela 🐱 ?

	Liste chainée	Tableau
Push	$O(1)$	$O(1)$ si pas de redim $O(n)$ si redim
Pop	$O(1)$	$O(1)$ si pas de redim $O(n)$ si redim
n push	$O(n)$	$O(n^2)$ 🤔 ??
n pop	$O(n)$	$O(n^2)$ 🤔 ??

Stack avec tableau: on raffine un peu l'analyse

Réfléchissons deux minutes: Quand est-ce qu'on fait un redimensionnement?

On double la taille du tableau quand on atteint la limite.

Sur n push, on fait donc un resize à ces positions:

- 1
- 2
- 4
- 8
- 16
- ...
- n

Donc sur les n -push, nous avons fait $\log_2(n)$ redimensionnements. Chaque redimensionnement m'a coûté maximum $\mathcal{O}(n)$ donc au total, les n -push sont en $\mathcal{O}(n \log(n))$.

Est-ce qu'on peut être plus précis ?

	Liste chaînée	Tableau
Push	$O(1)$	$O(1)$ si pas de redim $O(n)$ si redim
Pop	$O(1)$	$O(1)$ si pas de redim $O(n)$ si redim
n push	$O(n)$	$O(n^2)$ 🤔 ??
n pop	$O(n)$	$O(n^2)$ 🤔 ??

Stack avec tableau: on raffine un peu l'analyse

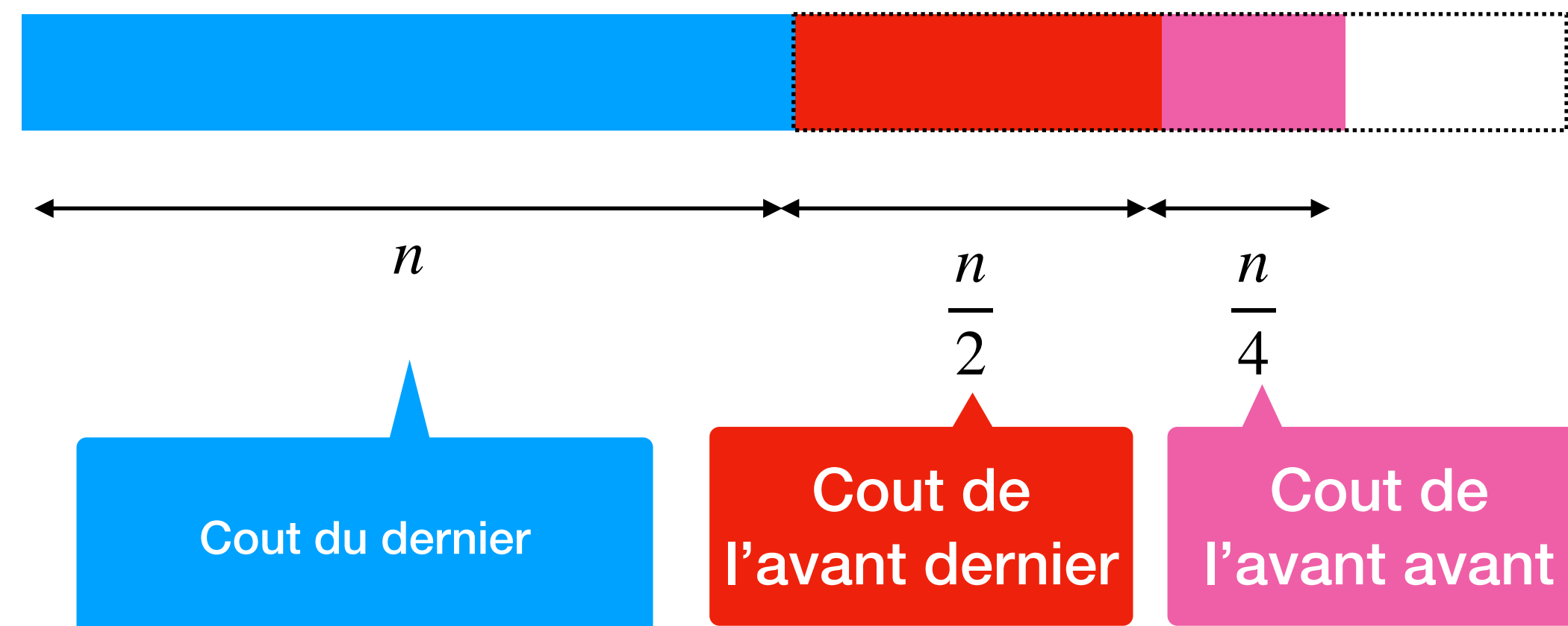
Réfléchissons deux minutes: Quand est-ce qu'on fait un redimensionnement?

	Liste chaînée	Tableau
Push	$O(1)$	$O(1)$ si pas de redim $O(n)$ si redim
Pop	$O(1)$	$O(1)$ si pas de redim $O(n)$ si redim
n push	$O(n)$	$O(n \log(n))$ 🤔 ??
n pop	$O(n)$	$O(n \log(n))$ 🤔 ??

On double la taille du tableau quand on atteint la limite.

Sur n push, on fait donc un resize à ces positions:

- 1
- 2
- 4
- 8
- 16
- ...
- n



Conclusion

java.util.Stack a de très bonnes complexités amorties pour n operations

	Liste chaînée	Tableau
Push	$O(1)$	$O(1)$ Sauf redim. $O(n)$
Pop	$O(1)$	$O(1)$ Sauf redim. $O(n)$
n push	$O(n)$	$O(n)$
n pop	$O(n)$	$O(n)$

On double la taille du tableau quand on atteint la limite.

Sur n push, on fait donc un resize à ces positions:

- 1
- 2
- 4
- 8
- 16
- ...
- n

ON PARLE DE COMPLEXITÉS AMORTIES

Combien de redim.?

$$\sum_{i=0}^{\log_2 n} \frac{n}{2^i} < 2n \in \mathcal{O}(n)$$

Question 1.1.4: Implémentation d'une pile (stack) avec deux files

- Rappel:

Stack API (LIFO)

push(e)

E pop()

Stack:

Last in, first out



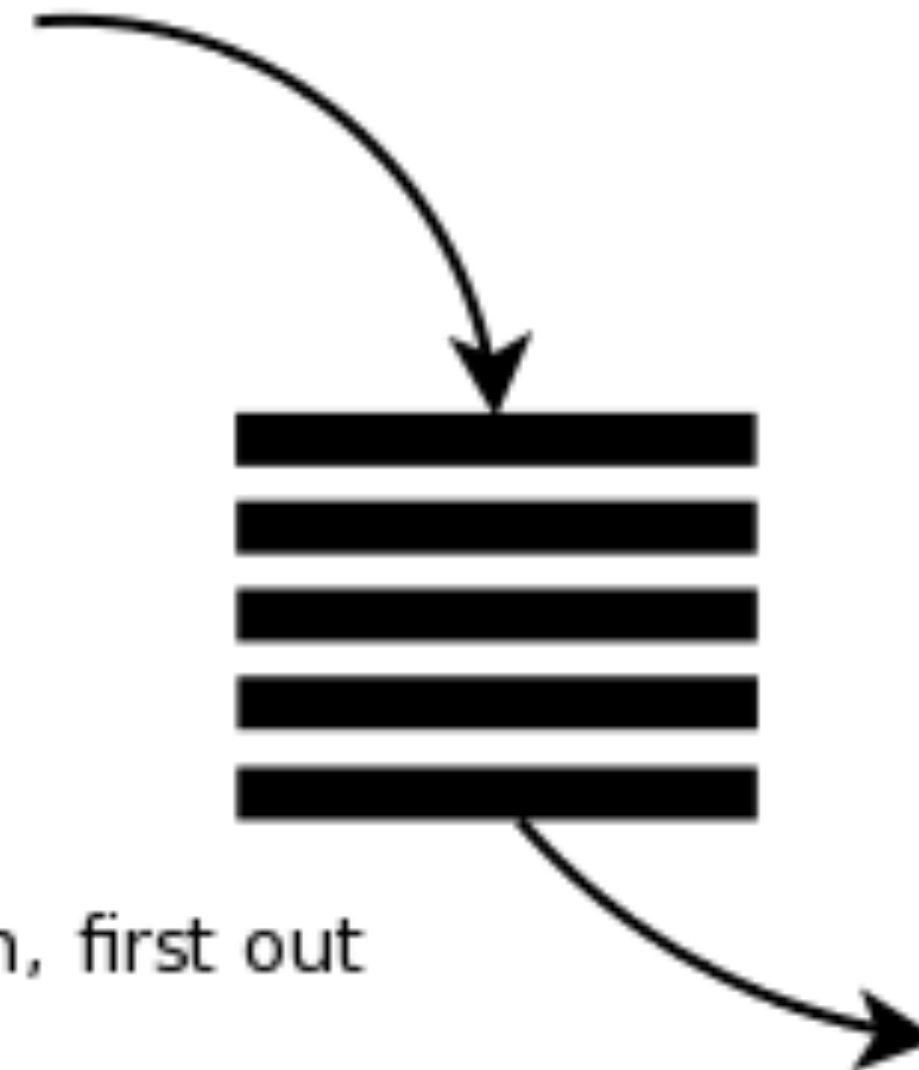
Queue API (FIFO)

enqueue(e)

E dequeue/remove()

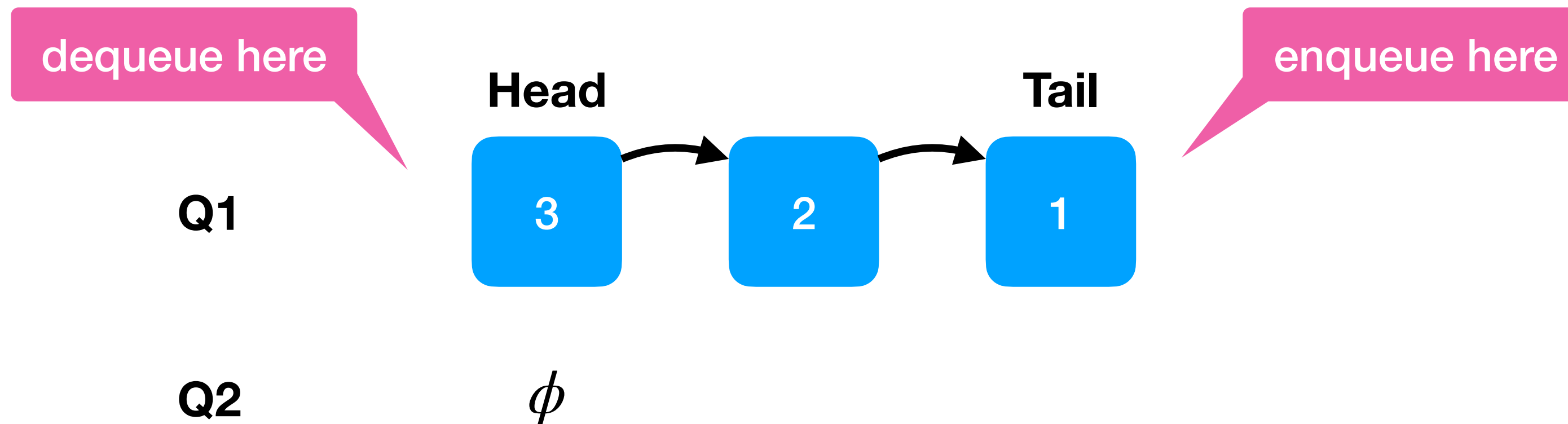
Queue:

First in, first out



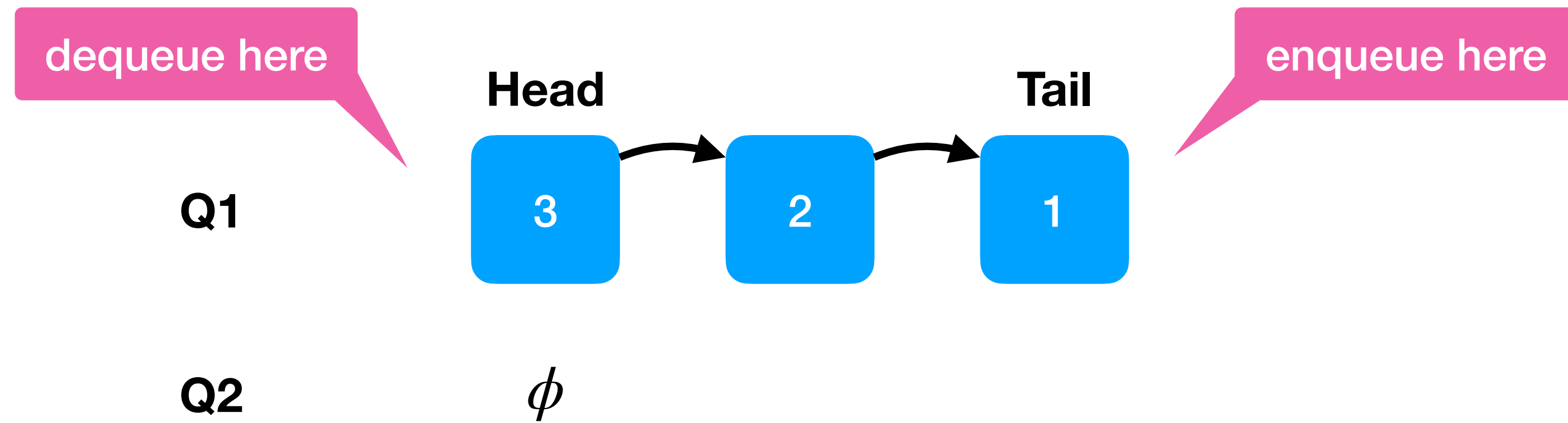
Question 1.1.4: L'idée = utilisation deux queue internes

- Soit Q1 et Q2 les deux queues internes
- Nous allons maintenir l'invariant que: *Q1 contains the elements of the stack from top (head) to the bottom (tail), Q2 is empty*
- Supposons que nous avons fait push(1), push(2), push(3). Notre invariant doit contenir

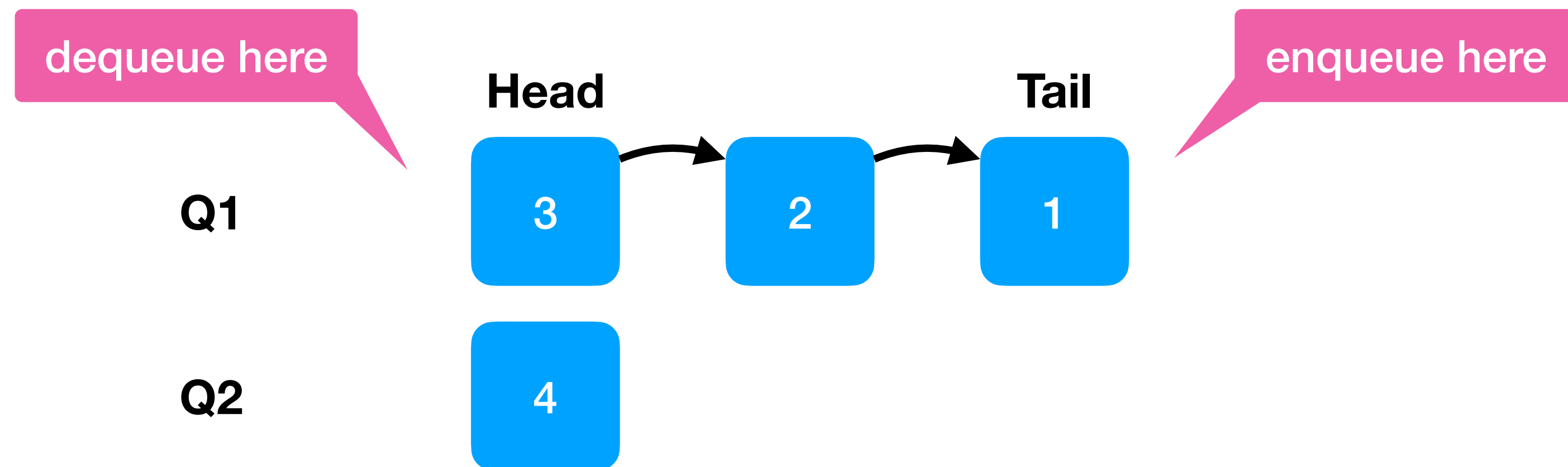


- Si je fais push(4) ensuite ... j'ajoute 4 (enqueue) dans Q2, et je vide Q1 dans Q2, puis j'échange les références.

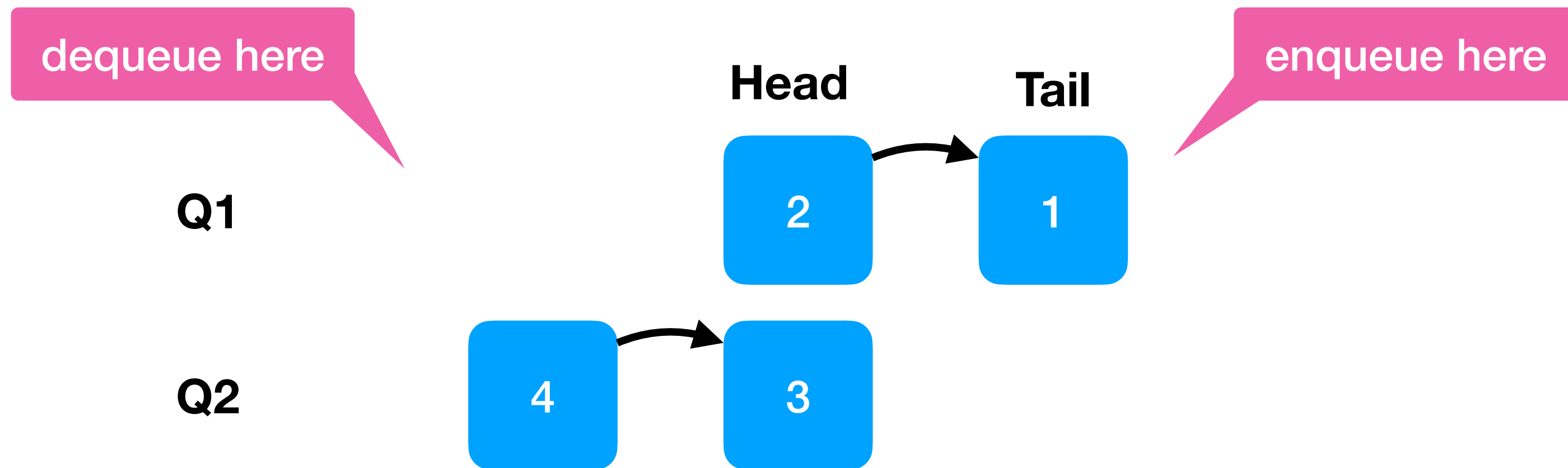
push(4)



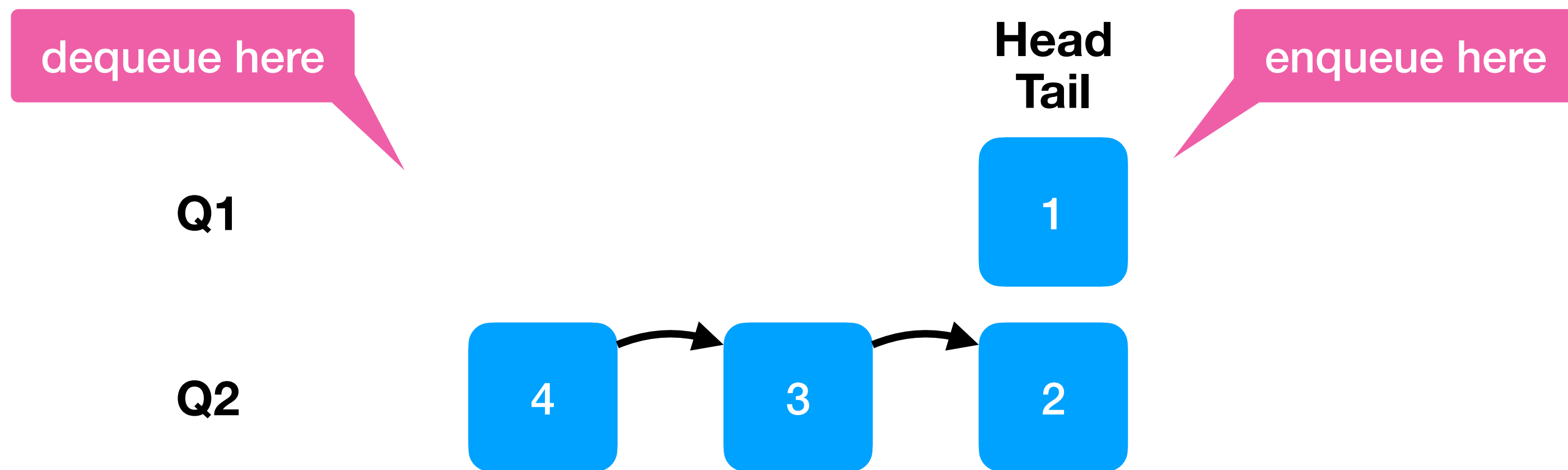
push(4): Q2.enqueue(4)



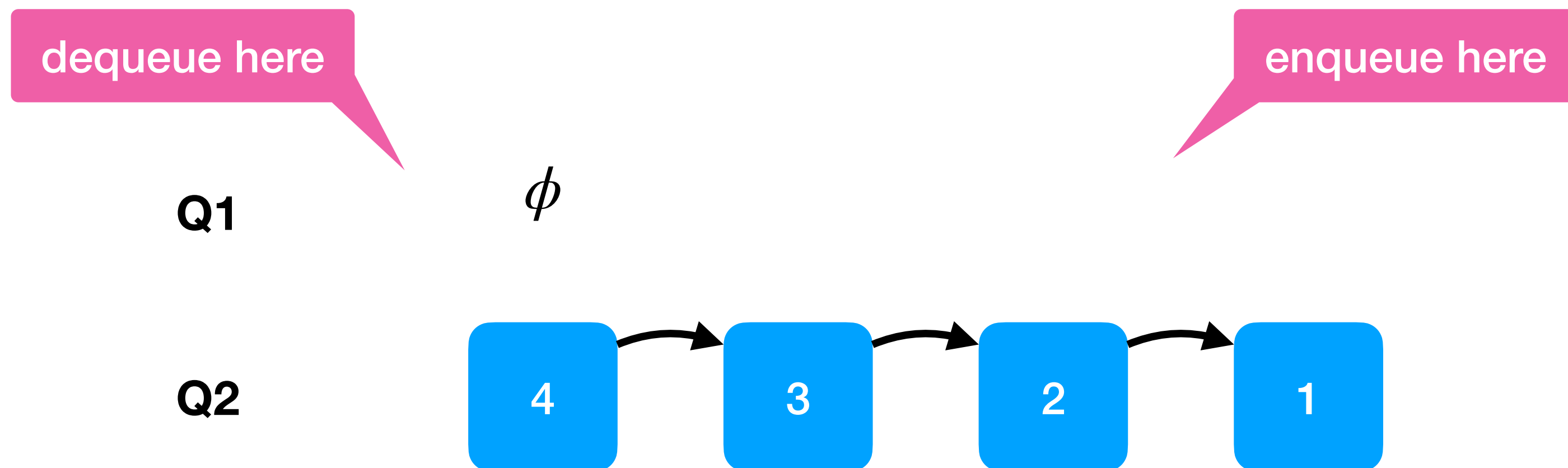
```
push(4): Q2.enqueue(Q1.dequeue())
```




```
push(4): Q2.enqueue(Q1.dequeue())
```

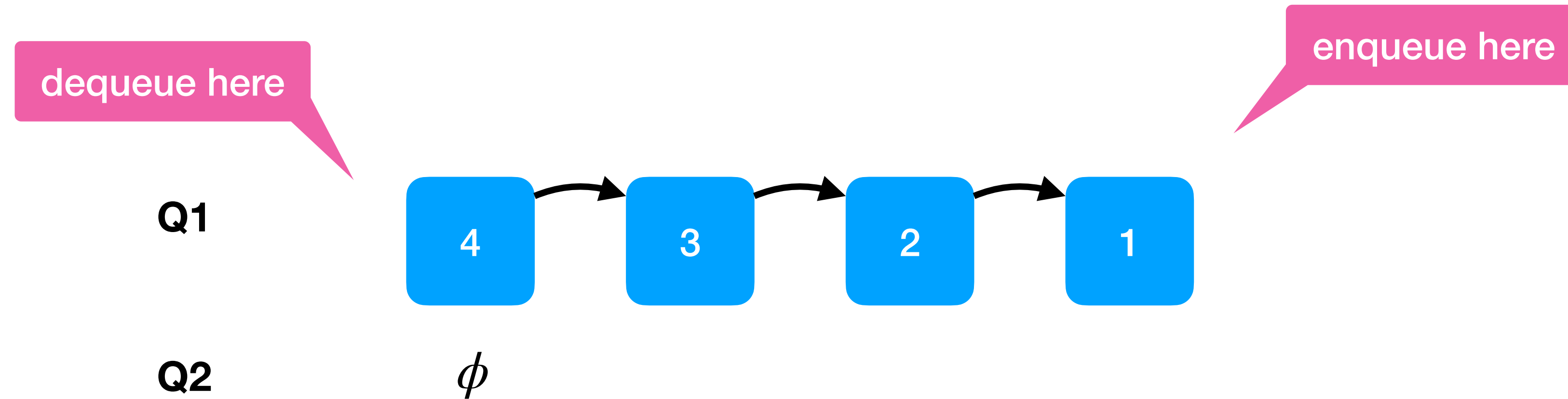


push(4): Q2.enqueue(Q1.dequeue())



push(4): exchange Q1 and Q2

- Invariant is established again!



Question 1.1.4: Pile avec deux files

```
Queue<E> queue1;  
Queue<E> queue2; // always empty  
  
public E pop() throws EmptyStackException {  
    if (empty()) throw new EmptyStackException();  
    return queue1.remove();  
}
```

```
public void push(E item) {  
    queue2.add(item);  
    while (!queue1.isEmpty()) {  
        queue2.add(queue1.remove()); // remove = dequeue  
    }  
    Queue<E> buffer = queue1;  
    queue1 = queue2;  
    queue2 = buffer;  
}
```

- Complexité push / pop ?

- $\Theta(n)$ for a push, $\mathcal{O}(1)$ for a pop

Question 1.1.5: Est-ce équivalent en terme de complexité ?

```
LinkedList<Integer> list = new LinkedList<>();
```

```
// assume I insert n elements in the list here
```

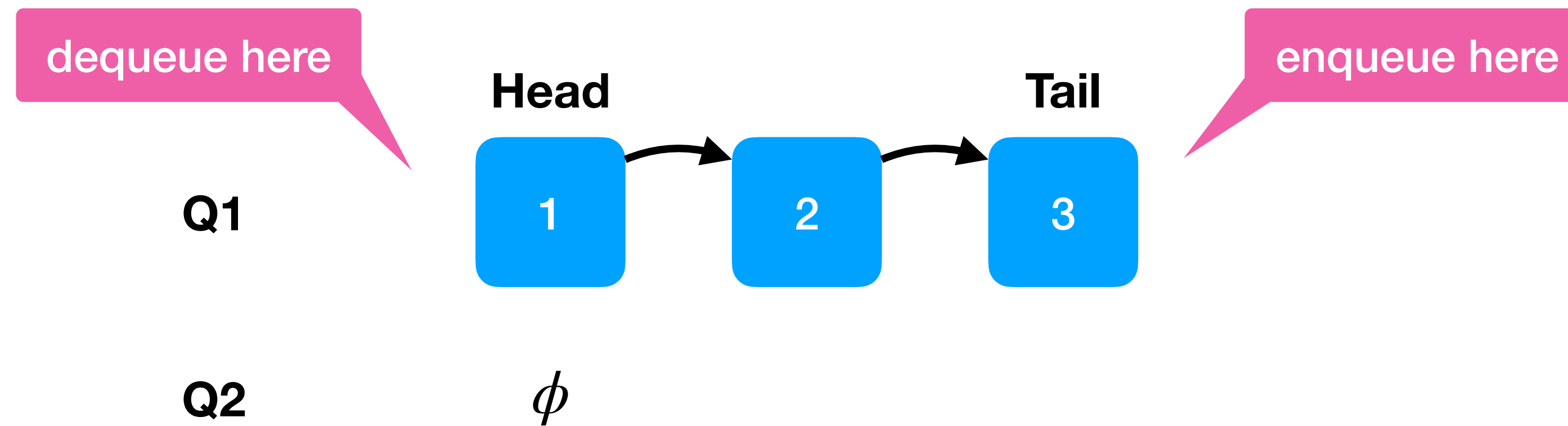
```
for (Integer val: list) {  
    System.out.println(val);  
}
```

```
Iterator<Integer> itr = list.iterator();  
while (itr.hasNext()) {  
    Integer val = itr.next();  
    System.out.println(val);  
}
```

```
for (int i = 0; i < list.size(); i++) {  
    Integer val = list.get(i);  
    System.out.println(val);  
}
```

Other solution

- Invariant: *Q1 contains the elements of the stack from bottom (head) to the top (tail), Q2 is empty*
 - Situation after push(1), push(2), push(3)



- Complexity push/pop ?
- $\mathcal{O}(1)$ for a push, $\Theta(n)$ for a pop

Question 1.1.5: Est-ce équivalent en terme de complexité ?

```
LinkedList<Integer> list = new LinkedList<>();
```

```
// assume I insert n elements in the list here
```

```
for (Integer val: list) {  
    System.out.println(val);  
}
```

```
Iterator<Integer> itr = list.iterator();  
while (itr.hasNext()) {  
    Integer val = itr.next();  
    System.out.println(val);  
}
```

```
for (int i = 0; i < list.size(); i++) {  
    Integer val = list.get(i);  
    System.out.println(val);  
}
```

Let's look into the byte code of

```
package fundamentals;
```

```
import java.util.LinkedList;
import java.util.List;
```

```
public class Test {
    public static void main(String[] args) {
        List<Integer> list = new LinkedList<>();
        list.add(1);
        list.add(2);
        for (Integer i : list) {
            System.out.println(i);
        }
    }
}
```

Decompile with javap -v

```
public static void main(java.lang.String[]);
```

```
descriptor: ([Ljava/lang/String;)V
```

```
flags: (0x0009) ACC_PUBLIC, ACC_STATIC
```

```
Code:
```

```
stack=2, locals=4, args_size=1
```

```
0: new          #7          // class java/util/LinkedList
```

```
3: dup
```

```
4: invokespecial #9          // Method java/util/LinkedList."<init>":()V
```

```
7: astore_1
```

```
8: aload_1
```

```
9: iconst_1
```

```
10: invokestatic #10          // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
```

```
13: invokeinterface #16, 2     // InterfaceMethod java/util/List.add:(Ljava/lang/Object;)Z
```

```
18: pop
```

```
19: aload_1
```

```
20: iconst_2
```

```
21: invokestatic #10          // Method java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
```

```
24: invokeinterface #16, 2     // InterfaceMethod java/util/List.add:(Ljava/lang/Object;)Z
```

```
29: pop
```

```
30: aload_1
```

```
31: invokeinterface #22, 1     // InterfaceMethod java/util/List.iterator:()Ljava/util/Iterator;
```

```
36: astore_2
```

```
37: aload_2
```

```
38: invokeinterface #26, 1     // InterfaceMethod java/util/Iterator.hasNext:()Z
```

```
43: ifeq         66
```

```
46: aload_2
```

```
47: invokeinterface #32, 1     // InterfaceMethod java/util/Iterator.next:()Ljava/lang/Object;
```

```
52: checkcast    #11          // class java/lang/Integer
```

```
55: astore_3
```

```
56: getstatic    #36          // Field java/lang/System.out:Ljava/io/PrintStream;
```

```
59: aload_3
```

```
60: invokevirtual #42          // Method java/io/PrintStream.println:(Ljava/lang/Object;)V
```

```
63: goto        37
```

```
66: return
```


Let's look into the implementation of LinkedList

```
for (int i = 0; i < list.size(); i++) {  
    Integer val = list.get(i);  
    System.out.println(val);  
}
```

```
public class LinkedList<E>  
    extends AbstractSequentialList<E>  
    implements List<E>, Deque<E>, Cloneable, java.io.Serializable  
{  
    transient int size = 0;  
  
    Pointer to first node.  
    transient Node<E> first;  
  
    Pointer to last node.  
    transient Node<E> last;
```

```
public E get(int index) {  
    checkElementIndex(index);  
    return node(index).item;  
}
```

Complexity ?

```
Node<E> node(int index) {  
    // assert isElementIndex(index);  
  
    if (index < (size >> 1)) {  
        Node<E> x = first;  
        for (int i = 0; i < index; i++)  
            x = x.next;  
        return x;  
    } else {  
        Node<E> x = last;  
        for (int i = size - 1; i > index; i--)  
            x = x.prev;  
        return x;  
    }  
}
```

Question 1.1.5: Est-ce équivalent en terme de complexité ?

```
LinkedList<Integer> list = new LinkedList<>();
```

How to change to get the complexity $O(n)$ also in the third case

```
// assume I insert n elements in the list here
```

```
for (Integer val: list) {  
    System.out.println(val);  
}
```

```
Iterator<Integer> itr = list.iterator();  
while (itr.hasNext()) {  
    Integer val = itr.next();  
    System.out.println(val);  
}
```

```
for (int i = 0; i < list.size(); i++) {  
    Integer val = list.get(i);  
    System.out.println(val);  
}
```

Question 1.1.6: types de complexités

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \text{ t.q. } f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$

« Appartient »
 $\mathcal{O}()$ est donc un
ensemble

Ceci est une fonction
qui compte le nombre
d'opérations en
fonction de n

Il existe un k tel que
 $k \cdot g(n)$ est plus grand
que $f(n)$ quand n est
grand

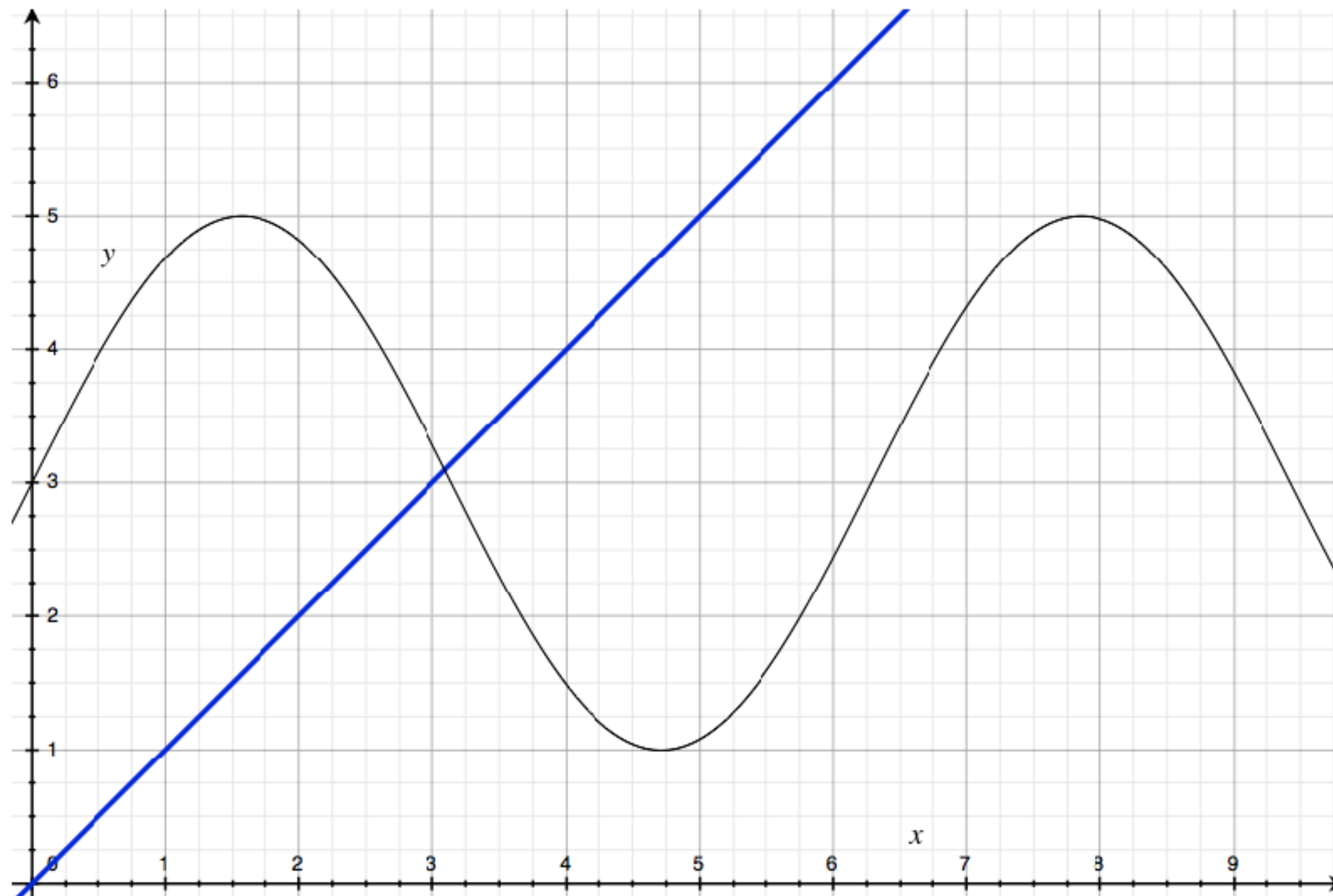
Question 1.1.6: TYPES de complexités

Le livre utilise la notation $\sim$ (tilde). On utilise dans d'autres cours les notations O , Ω et Θ .

Quelle est la différence entre ces différentes classes de complexités?

Question 1.1.6: types de complexités

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \text{ t.q. } f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$



$$f(x) = 2 \sin(x) + 2$$

$$g(x) = x$$

$$2 \sin(x) + 2 \in \mathcal{O}(x)$$

Question 1.1.6: types de complexités

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Omega}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad \mathbf{k} \cdot \mathbf{f(n)} \geq \mathbf{g(n)} \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Theta}(g(n)) \iff \exists k_0, k_1 \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad \mathbf{k_0} \cdot \mathbf{g(n)} \leq \mathbf{f(n)} \leq \mathbf{k_1} \cdot \mathbf{g(n)} \quad \forall n \geq n_0$$

Question 1.1.6: types de complexités

$$f(n) \in \mathcal{O}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad f(n) \leq k \cdot g(n) \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Omega}(g(n)) \iff \exists k \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad \mathbf{k} \cdot \mathbf{f(n)} \geq \mathbf{g(n)} \quad \forall n \geq n_0$$

$$f(n) \in \mathbf{\Theta}(g(n)) \iff \exists k_0, k_1 \in \mathbb{R}^+, n_0 \in \mathbb{N} \quad \text{t.q.} \quad \mathbf{k_0} \cdot \mathbf{g(n)} \leq \mathbf{f(n)} \leq \mathbf{k_1} \cdot \mathbf{g(n)} \quad \forall n \geq n_0$$

$$f(n) \sim g(n) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1$$

Question 1.1.7: Doubling RATIO TEST

- Rappel: A quoi sert le doubling ratio test ?
- En faisant l'hypothèse que la complexité d'un algorithme Si $f(n) \sim an^b \log n$ le test sert à trouver la valeur de l'exposant b juste en mesurant le temps d'exécution pour quelques valeur de taille d'entrée n , en doublant la taille à chaque fois (il n'y a même pas besoin de lire le code source!). On peut montrer que le ratio, à la limite converge vers:

$$\text{Si } f(n) \sim an^b \log n \quad \text{alors} \quad \frac{f(2n)}{f(n)} \sim 2^b$$

Question 1.1.7: Doubling RATIO TEST

n	1000	2000	4000	8000	16000	32000	64000
T(n)	0	0	0.1	0.3	1.3	5.1	20.5
Ratio précédent		~1	~1	~3	4.3	3.92	4.01

$\sim 2^b$ semble converger vers 4 sur cet exemple. Donc la complexité semble être
Si $f(n) \sim an^2[\log n]$ (en pratique, on ne sait pas s'il y a un log ou pas)